Tecnicatura Superior en Desarrollo de Software

Módulo 09 — Host de escritorio: `WPF` y Composition Root

Actividad Práctica — CU-CLI-001: Registrar Cliente | Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

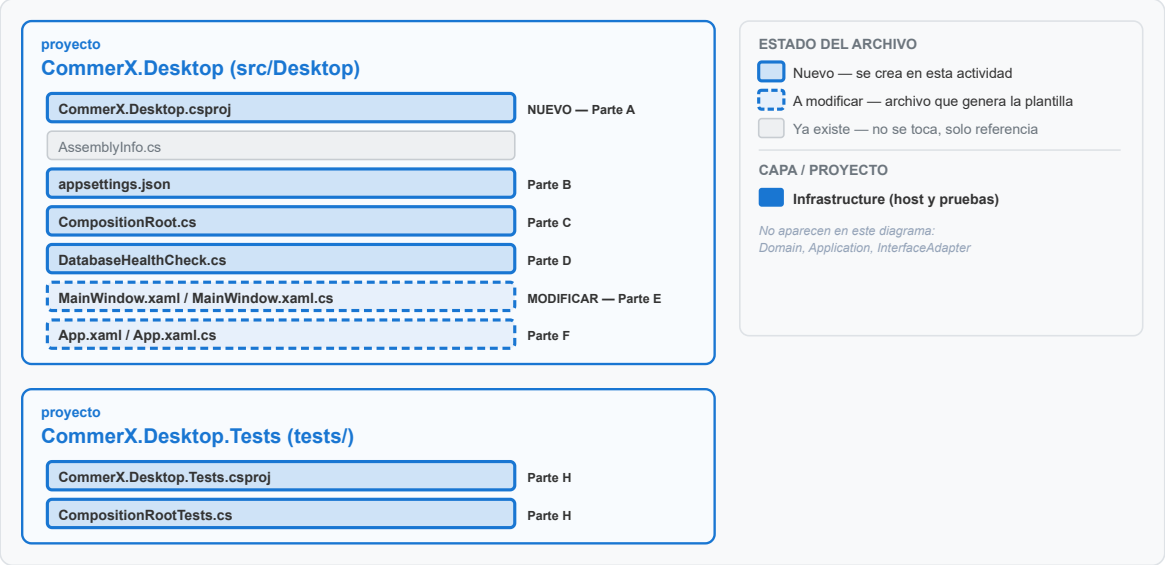
Contexto de la actividad

Hasta ahora construiste todas las piezas del caso de uso *Registrar Cliente*: el dominio, los casos de uso, el presenter, el ViewModel, la View y la persistencia en SQL Server. Pero ninguna de ellas **arranca** sola: son bibliotecas. Falta el programa que las reúne, las conecta entre sí y las pone a correr. Ese programa es el **host**, y el lugar donde arma las dependencias se llama *composition root* (raíz de composición).

En esta actividad construís el host de escritorio como una aplicación **WPF** que usa el host genérico de .NET (`Microsoft.Extensions.Hosting`): lee la configuración, arma el contenedor de dependencias con todas las capas y muestra una ventana que comprueba que la base de datos responde. La pantalla de alta de clientes (la View del Módulo 7) todavía **no** se muestra: eso se agrega en el Módulo 10 con Blazor Hybrid. Separarlo así tiene una ventaja práctica: si algo falla, sabés si el problema es del host o de Blazor.

> **Punto de partida:** la solución tal como quedó al terminar el Módulo 8, incluido el ejercicio de la Parte H (`UpdateCustomerPresenter` y su registro). Hace falta porque el host registra todos los presenters. La base de datos `CommerX` ya tiene que existir (migración del Módulo 8).

Archivos involucrados



El host es la capa más externa: conoce a todas las demás y ninguna lo conoce a él. Por eso vive dentro de la carpeta de solución `4.Infrastructure`. Los archivos con borde sólido son **NUEVOS**, los de borde discontinuo se **MODIFICAN** (los genera la plantilla de WPF y se reemplazan) y el gris lo genera la plantilla y no se toca.

Parte A — Crear el proyecto WPF, sus referencias y su paquete

Ubicación

`src/Desktop/` (proyecto nuevo, dentro de la carpeta de solución `4.Infrastructure`)

Descripción

WPF (*Windows Presentation Foundation*) es el sistema de ventanas de escritorio de Windows. Una aplicación WPF es un proyecto con una forma propia: un archivo `App.xaml` que representa a la aplicación y archivos `.xaml` que describen cada ventana. **XAML** es un lenguaje de marcado, parecido a HTML, para describir una interfaz; cada archivo `.xaml` tiene un archivo C# pareja (`.xaml.cs`, el *code-behind*) con la lógica de esa ventana.

Paso 1 — Crear el proyecto

Opción 1 — Línea de comandos (desde la carpeta raíz de la solución, donde está `CommerX.slnx`):

```
dotnet new wpf -n CommerX.Desktop -o src/Desktop -f net9.0
dotnet sln CommerX.slnx add src/Desktop/CommerX.Desktop.csproj --solution-folder 4.Infrastructure
```

1. **Comando 1:** `dotnet new wpf` crea un proyecto desde la plantilla de WPF. `-n` es el nombre, `-o` la carpeta y `-f net9.0` el framework (el mismo que el resto de la solución).
2. **Comando 2:** lo agrega a la solución dentro de la carpeta `4.Infrastructure`.

Opción 2 — Visual Studio: clic derecho sobre la carpeta `4.Infrastructure` → **Agregar** → **Nuevo proyecto…** → plantilla **Aplicación WPF** (la de C# para .NET, no la de .NET Framework) → nombre `CommerX.Desktop`, ubicación la carpeta `src` → framework **.NET 9.0**.

Paso 2 — Qué generó la plantilla

Abrí el archivo `CommerX.Desktop.csproj`. Esto es lo que trae:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net9.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseWPF>true</UseWPF>
  </PropertyGroup>

</Project>
```

- **`OutputType` = `WinExe`:** el resultado es un ejecutable de Windows sin ventana de consola. (Las bibliotecas que hiciste hasta ahora no tienen esta línea: producen `.dll`, no un programa.)
- **`TargetFramework` = `net9.0-windows`:** el sufijo `-windows` indica que usa APIs exclusivas de Windows. WPF no existe en otros sistemas.
- **`UseWPF` = `true`:** activa el compilador de XAML y las bibliotecas de WPF.

La plantilla también creó cinco archivos: `App.xaml` y `App.xaml.cs` (la aplicación), `MainWindow.xaml` y `MainWindow.xaml.cs` (la ventana inicial) y `AssemblyInfo.cs` (un atributo de la plantilla que no se toca). Los cuatro primeros se reemplazan en las Partes E y F.

Paso 3 — Referenciar los proyectos

El host es el único proyecto que conoce a todas las capas, porque es el que las conecta. Necesita referenciar a InterfaceAdapter (de ahí llegan, por transitividad, Application y Domain) y a Infrastructure:

```
dotnet add src/Desktop/CommerX.Desktop.csproj reference src/InterfaceAdapter/CommerX.InterfaceAdapter.csproj src/Infrastructure/CommerX.Infrastructure.csproj
```

En Visual Studio: clic derecho sobre **Dependencias** del proyecto `CommerX.Desktop` → **Agregar referencia de proyecto…** → marcá `CommerX.InterfaceAdapter` y `CommerX.Infrastructure`. No hace falta referenciar a Application ni a Domain: las referencias entre proyectos son *transitivas*, así que llegan solas a través de los otros dos.

Paso 4 — Instalar el paquete del host genérico

`Microsoft.Extensions.Hosting` es el paquete NuGet que trae el **host genérico** de .NET: la clase `Host` con la que se arma una aplicación que ya incluye configuración (los archivos `appsettings`), el contenedor de inyección de dependencias y registro de eventos (logging). Igual que con Entity Framework Core, se instala con versión exacta de la rama 9.x, porque la solución apunta a `net9.0`:

```
dotnet add src/Desktop/CommerX.Desktop.csproj package Microsoft.Extensions.Hosting --version 9.0.20
```

En Visual Studio: clic derecho sobre el proyecto → **Administrar paquetes NuGet…** → pestaña **Examinar** → `Microsoft.Extensions.Hosting` → versión `9.0.20` → **Instalar**.

Paso 5 — Copiar appsettings.json junto al ejecutable

El archivo `appsettings.json` (Parte B) tiene que estar al lado del programa cuando se ejecuta, pero por defecto un archivo del proyecto *no* se copia a la carpeta de salida. Se le pide con una línea en el `.csproj`. Así queda el archivo completo, con los pasos 3, 4 y 5:

```
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <ItemGroup>
4      <ProjectReference Include="..\InterfaceAdapter\CommerX.InterfaceAdapter.csproj" />
5      <ProjectReference Include="..\Infrastructure\CommerX.Infrastructure.csproj" />
6    </ItemGroup>
7
8    <ItemGroup>
9      <PackageReference Include="Microsoft.Extensions.Hosting" Version="9.0.20" />
10   </ItemGroup>
11
12   <ItemGroup>
13     <!-- appsettings.json se copia junto al ejecutable para que el host lo encuentre al arrancar -->
14     <None Update="appsettings.json">
15       <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
16     </None>
17   </ItemGroup>
18
19   <PropertyGroup>
20     <OutputType>WinExe</OutputType>
21     <TargetFramework>net9.0-windows</TargetFramework>
22     <Nullable>enable</Nullable>
23     <ImplicitUsings>enable</ImplicitUsings>
24     <UseWPF>true</UseWPF>
25   </PropertyGroup>
26
27 </Project>
```

Cómo leerlo:
Líneas 4 y 5: las dos referencias a proyecto (paso 3).

Línea 9: el paquete del host genérico, con versión exacta (paso 4).

Líneas 14 a 16: le indica a MSBuild que copie `appsettings.json` a la carpeta

de salida cada vez que cambie (`PreserveNewest`). Sin esto, la aplicación arranca sin encontrar su configuración.

> **Comprobá:** todavía no existe `appsettings.json` , pero eso no impide compilar. Ejecutá `dotnet build src/Desktop/CommerX.Desktop.csproj` : tiene que terminar con 0 errores y 0 advertencias.

Parte B — Crear `appsettings.json`

Ubicación

`src/Desktop/appsettings.json`

Descripción

Los datos que cambian según dónde se ejecuta la aplicación (la dirección del servidor de base de datos, la contraseña, etc.) **no se escriben en el código**: se leen de un archivo de configuración. Este es el archivo estándar de .NET, en formato JSON. Creá un archivo nuevo con ese nombre exacto en la raíz del proyecto `src/Desktop` :

```
1  {
2    "ConnectionStrings": {
3      "CommerX": "Server=(localdb)\\MSSQLLocalDB;Database=CommerX;Trusted_Connection=True;"
4    }
5  }
```

Qué está pasando:
Línea 2: `ConnectionStrings` es el nombre de sección que el host genérico espera para las cadenas de conexión: así después se pueden leer con `GetConnectionString("CommerX")` .

Línea 3: la cadena de conexión. `(localdb)\MSSQLLocalDB` es SQL Server LocalDB (el que usaste para la migración del Módulo 8); en JSON la barra invertida se escribe **doble** (`\\`) porque es el carácter de escape. `Database=CommerX` es la base de datos y `Trusted_Connection=True` usa tu usuario de Windows para autenticarse, sin contraseña.

> **Si tu servidor no es LocalDB:** cambiá el valor de `Server` y, si tiene un certificado de desarrollo, agregá `TrustServerCertificate=True;` (igual que en el Módulo 8). Para evitar que una contraseña quede escrita en este archivo, mirá el anexo 09a.

Parte C — Crear `CompositionRoot.cs`

Ubicación

`src/Desktop/CompositionRoot.cs`

Descripción

Cada capa ya tiene su método de registro (`AddCustomerUseCases`, `AddCustomerPresenter`, `AddInfrastructure`…). El *composition root* es el único lugar que los llama todos. Se escribe como un método de extensión propio del host, `AddCommerX`, para que sea fácil de probar sin abrir ninguna ventana (Parte H).

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace CommerX.Desktop;

// composition root — el único lugar de la aplicación que conoce todas las capas y arma las dependencias
public static class CompositionRoot
{
    public static IServiceCollection AddCommerX(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        // la cadena de conexión viene de appsettings.json; si falta, se falla al arrancar con un mensaje claro
        var connectionString = configuration.GetConnectionString("CommerX")
            ?? throw new InvalidOperationException(
                "Falta la cadena de conexión 'CommerX' en appsettings.json (sección ConnectionStrings).");

        // Application: casos de uso y validación
        services.AddCustomerUseCases();

```

```
21          // InterfaceAdapter: presenters y view models
22          services.AddCustomerPresenter();
23          services.AddUpdateCustomerPresenter();
24          services.AddCustomerViewModel();
25
26          // Infrastructure: DbContext y repositorios
27          services.AddInfrastructure(connectionString);
28
29          // servicios propios del host
30          services.AddSingleton<DatabaseHealthCheck>();
31          services.AddSingleton<MainWindow>();
32
33          return services;
34      }
35  }
```

Qué está pasando:
Líneas 14 a 16: lee la cadena de conexión de la configuración. Si falta, la aplicación falla *al arrancar* con un mensaje que dice qué falta y dónde, en lugar de fallar más tarde, en un lugar lejano y con un error confuso.

Líneas 18 a 27: registra cada capa de adentro hacia afuera: casos de uso (Application), presenters y ViewModel (InterfaceAdapter) y persistencia (Infrastructure). Fijate que se registra también el presenter de actualización: el caso de uso de CU-CLI-002 lo necesita, y por eso la Parte H del Módulo 8 era requisito.

Líneas 30 y 31: los dos servicios propios del host: el chequeo de base de datos y la ventana principal. La ventana también se registra en el contenedor, así recibe sus dependencias por constructor (Parte E).

Parte D — Crear `DatabaseHealthCheck.cs`

Ubicación

`src/Desktop/DatabaseHealthCheck.cs`

Descripción

Una clase mínima que responde una sola pregunta: "¿la base de datos responde?". Sirve para que la ventana del host muestre el estado, y de paso enseña algo importante sobre los *tiempos de vida* de los servicios en una aplicación de escritorio.

```
1  using CommerX.Infrastructure.Data;
2  using Microsoft.EntityFrameworkCore;
3  using Microsoft.Extensions.DependencyInjection;
4
5  namespace CommerX.Desktop;
6
7  // comprueba que la base de datos responde; vive como Singleton y por eso abre su
   propio scope
8  public sealed class DatabaseHealthCheck
9  {
10     private readonly IServiceScopeFactory _scopeFactory;
11
12     public DatabaseHealthCheck(IServiceScopeFactory scopeFactory)
13     {
14         _scopeFactory = scopeFactory;
15     }
16
17     public async Task<bool> CanConnectAsync()
18     {
19         // el DbContext es Scoped: un Singleton no puede recibirlo por constructo
   r, así que se crea un scope
20         using var scope = _scopeFactory.CreateScope();
21         var context = scope.ServiceProvider.GetRequiredService<CommerXDbContext>
   ();
22
23         return await context.Database.CanConnectAsync();
24     }
25 }
```

Qué está pasando:
Línea 12: la clase recibe un `IServiceScopeFactory`, la "fábrica de scopes" del contenedor, y no el `CommerXDbContext` directamente.

Por qué: `CommerXDbContext` está registrado como *Scoped* (vive mientras dura una unidad de trabajo). En una aplicación web, cada pedido abre un scope solo. En una aplicación de escritorio **nadie abre scopes por vos**. Esta clase vive mientras vive la aplicación (*Singleton*) y un Singleton no puede recibir un Scoped

por constructor: el contenedor lo rechaza con *"Cannot consume scoped service ... from singleton"*.

Líneas 20 a 21: la solución es abrir un scope cuando hace falta, pedir el contexto adentro y cerrarlo al terminar (`using`). **Línea 23:** `CanConnectAsync` devuelve verdadero o falso; no lanza una excepción si el servidor no responde.

> **Una regla práctica:** un servicio de vida larga (Singleton) que necesita algo de vida corta (Scoped) recibe la fábrica de scopes y abre uno por operación. Lo vas a volver a ver en el Módulo 10, donde `OwningComponentBase` hace lo mismo por cada pantalla. El anexo 04c explica los tres tiempos de vida.

Parte E — Reemplazar `MainWindow.xaml` y su code-behind

Ubicación

`src/Desktop/MainWindow.xaml` y `src/Desktop/MainWindow.xaml.cs` (los generó la plantilla; reemplazá todo su contenido)

Descripción

La ventana tiene un título, un texto de estado y un botón. Primero el diseño, en XAML:

```
<Window x:Class="CommerX.Desktop.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="CommerX" Height="260" Width="480"
        WindowStartupLocation="CenterScreen"
        Loaded="OnLoaded">
    <StackPanel Margin="24" VerticalAlignment="Center">
        <TextBlock Text="CommerX" FontSize="28" FontWeight="Bold" />
        <TextBlock x:Name="StatusText" Text="Base de datos: sin comprobar" Margin="0,12,0,12" FontSize="14" />
        <Button Content="Comprobar conexión" Click="OnCheckConnection" Padding="12,6" HorizontalAlignment="Left" />
    </StackPanel>
</Window>
```

Qué está pasando:

Línea 4: el título de la ventana, su tamaño y su posición al abrir.

Línea 6: `Loaded="OnLoaded"` enlaza el evento "la ventana terminó de cargarse" con el método `OnLoaded` del code-behind.

Línea 9: `x:Name` le pone un nombre al texto de estado, para poder cambiarlo desde C#.

Línea 10: `Click="OnCheckConnection"` enlaza el botón con su método.

Y ahora la lógica, en el code-behind:

```
1  using System.Windows;
2
3  namespace CommerX.Desktop;
4
5  public partial class MainWindow : Window
6  {
7      private readonly DatabaseHealthCheck _healthCheck;
8
9      // el contenedor inyecta la dependencia: la ventana no sabe cómo se comprueba
   la base
10     public MainWindow(DatabaseHealthCheck healthCheck)
11     {
12         _healthCheck = healthCheck;
13         InitializeComponent();
14     }
15
16     private async void OnLoaded(object sender, RoutedEventArgs e) => await CheckA
   sync();
17
18     private async void OnCheckConnection(object sender, RoutedEventArgs e) => awa
   it CheckAsync();
19
20     private async Task CheckAsync()
21     {
22         StatusText.Text = "Base de datos: comprobando...";
23
24         var connected = await _healthCheck.CanConnectAsync();
25
26         StatusText.Text = connected
27             ? "Base de datos: conectada"
28             : "Base de datos: sin conexión";
29     }
30 }
```

Qué está pasando:
Línea 10: el constructor recibe el `DatabaseHealthCheck`: la ventana no sabe cómo se comprueba la base, solo lo pide. Quien se lo da es el contenedor, porque la ventana está registrada en él (Parte C) y se resuelve desde él (Parte F).

Línea 13: `InitializeComponent()` construye los controles que se describieron en el XAML. Es obligatorio en todo code-behind de WPF.

Líneas 16 y siguientes: los dos manejadores de eventos son `async void`. Es el único caso donde `async void` es correcto: los eventos de la interfaz no devuelven nada. Los dos delegan en `CheckAsync` (línea 20), que sí devuelve `Task` y es el que contiene la lógica.

Parte F — Reemplazar `App.xaml` y `App.xaml.cs`

Ubicación

`src/Desktop/App.xaml` y `src/Desktop/App.xaml.cs` (los generó la plantilla; reemplazalos)

Descripción

`App` es la clase que representa a toda la aplicación y donde se arma el host. Empezamos por el XAML. La plantilla trae una línea que hay que **quitar**:

```
1  <Application x:Class="CommerX.Desktop.App"
2               xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3               xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4               xmlns:local="clr-namespace:CommerX.Desktop"
5               StartupUri="MainWindow.xaml">
```

`StartupUri="MainWindow.xaml"` hace que WPF cree la ventana principal por su cuenta, con `new MainWindow()`. Eso se saltea el contenedor de dependencias: la ventana no recibiría su `DatabaseHealthCheck` y la aplicación fallaría al abrirse. Hay que quitar esa línea y mostrar la ventana desde el código, resolviéndola del contenedor. Este es el `App.xaml` resultante:

```
1  <Application x:Class="CommerX.Desktop.App"
```

```
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Application.Resources>

    </Application.Resources>
</Application>
```

Y el code-behind, que es donde se construye y se arranca el host:

```
using System.Windows;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

namespace CommerX.Desktop;

// se califica System.Windows.Application: sin eso choca con el namespace CommerX.Application
public partial class App : System.Windows.Application
{
    private readonly IHost _host;

    public App()
    {
        // el host genérico trae configuración (appsettings.json), inyección de dependencias y logging
        _host = Host.CreateDefaultBuilder()
            // appsettings.json se busca junto al ejecutable, sin importar desde dónde se lance
            .UseContentRoot(AppContext.BaseDirectory)
            .ConfigureServices((context, services) => services.AddCommerX(context.Configuration))
            .Build();
    }

    protected override async void OnStartup(StartupEventArgs e)
    {
        await _host.StartAsync();

        // la ventana se resuelve del contenedor, no se crea con new: así recibe sus dependencias
        _host.Services.GetRequiredService<MainWindow>().Show();

        base.OnStartup(e);
    }

    protected override async void OnExit(ExitEventArgs e)
    {
        await _host.StopAsync();
```

```
35          _host.Dispose();
36
37          base.OnExit(e);
38      }
39  }
```

Qué está pasando:
Línea 8: la clase base se escribe `System.Windows.Application`, completo. La plantilla dice solo `Application`, pero ahora el proyecto referencia a `CommerX.Application` y el nombre `Application` pasa a significar ese *espacio de nombres*: sin calificarlo el compilador falla con *"'Application' es espacio de nombres pero se usa como tipo"*.

Línea 10: el host se guarda en un campo para poder arrancarlo y detenerlo.

Líneas 15 a 19: `Host.CreateDefaultBuilder()` crea el host con la configuración estándar: carga `appsettings.json`, las variables de entorno, el contenedor de dependencias y el logging.

Línea 17: `UseContentRoot(AppContext.BaseDirectory)` hace que el host busque `appsettings.json` junto al ejecutable. Sin esta línea lo busca en el directorio desde el que se lanzó el programa, y si lo abrís haciendo doble clic desde otro lugar no encuentra la configuración.

Línea 18: aquí se llama al composition root de la Parte C, pasándole la configuración.

Líneas 22 a 27: al arrancar, se inicia el host y se *resuelve la ventana del contenedor* (línea 27), no se crea con `new`; después se muestra.

Líneas 32 en adelante: al cerrar la aplicación se detiene el host y se liberan sus recursos.

> **Por qué `async void` en `OnStartup` y `OnExit`:** son métodos que WPF ya define sin valor de retorno y se sobrescriben tal cual; es el mismo caso de los manejadores de eventos de la Parte E.

Parte G — Ejecutar la aplicación y comprobar la base de datos

Descripción

Ahora sí: se compila y se ejecuta el host. La base de datos `CommerX` tiene que existir (la creaste con la migración del Módulo 8). Si no existe, o la borraste, volvé a crearla:

```
cd src/Infrastructure
dotnet ef database update
cd ../..
```

Después ejecutá el host, desde la carpeta raíz de la solución:

```
dotnet run --project src/Desktop/CommerX.Desktop.csproj
```

En Visual Studio podés lo mismo con **F5**, dejando `CommerX.Desktop` como proyecto de inicio (clic derecho sobre el proyecto → **Establecer como proyecto de inicio**).

> **Resultado esperado:** se abre una ventana titulada **CommerX** que primero dice *"Base de datos: comprobando..."* y enseguida **"Base de datos: conectada"**. El botón *Comprobar conexión* repite la prueba.

Comprobar también el caso de error

Cerrá la ventana. En `src/Desktop/appsettings.json` cambiá `Database=CommerX` por `Database=CommerXX` (una base que no existe) y ejecutá de nuevo: ahora la ventana tiene que decir **"Base de datos: sin conexión"**. Volvé a dejar `Database=CommerX`. Fijate que la aplicación **no se rompe** cuando la base no responde: informa el problema, y eso es lo que `CanConnectAsync` permite.

> **Si la aplicación se cierra al abrir:** lo más probable es que falte la cadena de conexión o que `appsettings.json` no se haya copiado a la carpeta de salida (revisá el paso 5 de la Parte A). El mensaje del error dice cuál de las dos cosas pasa.

Parte H — Probar el composition root sin abrir ninguna ventana

Ubicación

`tests/CommerX.Desktop.Tests/` (proyecto de pruebas nuevo, junto al host en la carpeta `4.Infrastructure`)

Descripción

Antes de empezar: qué es una prueba automática. Hasta ahora comprobaste que el código funcionaba ejecutándolo y mirando el resultado a mano. Una *prueba automática* hace lo mismo, pero es código: ejecuta una parte de tu programa y verifica por vos que el resultado sea el esperado. Así se lee una prueba:

- `[Fact]` sobre un método le dice a xUnit (la herramienta que corre las pruebas) que ese método es una prueba.
- `Assert` compara lo que esperabas con lo que realmente pasó. Si coinciden, la prueba pasa; si no, falla.
- **Verde** significa que todas las pruebas pasaron; **rojo**, que alguna falló y el mensaje te dice cuál y por qué.

No necesitás dominar el tema: en esta parte solo vas a copiar las pruebas, correrlas y leer el resultado. Fijate que acá no se prueba la lógica del negocio, sino que el host esté bien armado (que cada pieza tenga todo lo que necesita para crearse).

Abrir la ventana comprueba que el host funciona *en tu máquina*, pero es un paso manual. Lo que más se rompe en un host es la composición (un servicio sin registrar, un Scoped dentro de un Singleton), y eso se puede comprobar con una prueba automática. Para probar código de un proyecto WPF, el proyecto de pruebas tiene que apuntar al mismo framework `net9.0-windows`. Creá el archivo `CommerX.Desktop.Tests.csproj` con este contenido:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net9.0-windows</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
```

```
    <IsPackable>false</IsPackable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.2" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\..\src\Desktop\CommerX.Desktop.csproj" />
  </ItemGroup>

</Project>
```

Qué está pasando: un proyecto de pruebas es un proyecto más de la solución, con su propio `.csproj`, que no forma parte de la aplicación: existe solo para ejecutar código de otros proyectos y verificarlo. Este usa xUnit y tiene dos particularidades: `net9.0-windows` como framework (para poder referenciar al host) y una única referencia a proyecto, al host `CommerX.Desktop`. Agregalo a la solución:

```
dotnet sln CommerX.slnx add tests/CommerX.Desktop.Tests/CommerX.Desktop.Tests.csproj --solution-folder 4.Infrastructure
```

Y creá el archivo de pruebas:

```
using CommerX.Application.Customers.Ports;
using CommerX.Domain.Customers.Repositories;
using CommerX.Infrastructure.Data;
using CommerX.InterfaceAdapter.Customers.Presenters;
using CommerX.InterfaceAdapter.Customers.ViewModels;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace CommerX.Desktop.Tests;

public sealed class CompositionRootTests
{
```

```
14      private const string LocalDb = "Server=(localdb)\\MSSQLLocalDB;Database=Comme
    rX_Tests;Trusted_Connection=True;";
15
16      private static IConfiguration ConfigurationWith(string? connectionString)
17      {
18          var values = new Dictionary<string, string?>();
19
20          if (connectionString is not null)
21          {
22              values["ConnectionStrings:CommerX"] = connectionString;
23          }
24
25          return new ConfigurationBuilder().AddInMemoryCollection(values).Build();
26      }
27
28      // las mismas dos opciones de validación estricta que conviene usar para dete
    ctar errores de configuración
29      private static ServiceProvider Build(IConfiguration configuration)
30      {
31          var services = new ServiceCollection();
32          services.AddCommerX(configuration);
33
34          return services.BuildServiceProvider(new ServiceProviderOptions
35          {
36              ValidateOnBuild = true,
37              ValidateScopes = true
38          });
39      }
40
41      [Fact]
42      public void AddCommerX_WithoutTheConnectionString_FailsWithAClearMessage()
43      {
44          var services = new ServiceCollection();
45
46          var error = Assert.Throws<InvalidOperationException>(
47              () => services.AddCommerX(ConfigurationWith(null)));
48
49          Assert.Contains("CommerX", error.Message);
50          Assert.Contains("appsettings.json", error.Message);
51      }
52
53      [Fact]
54      public void AddCommerX_BuildsAContainerWhereEveryRegistrationCanBeResolved()
55      {
56          // ValidateOnBuild revisa que cada servicio registrado tenga todas sus de
    pendencias registradas
57          using var provider = Build(ConfigurationWith(LocalDb));
58
```

```
59          Assert.NotNull(provider);
60      }
61
62      [Fact]
63      public void AddCommerX_ResolvesTheWholeChainOfACaseOfUseInsideAScope()
64      {
65          using var provider = Build(ConfigurationWith(LocalDb));
66          using var scope = provider.CreateScope();
67
68          Assert.NotNull(scope.ServiceProvider.GetRequiredService<ICreateCustomerInputPort>());
69          Assert.NotNull(scope.ServiceProvider.GetRequiredService<IUpdateCustomerInputPort>());
70          Assert.NotNull(scope.ServiceProvider.GetRequiredService<CreateCustomerViewModel>());
71          Assert.NotNull(scope.ServiceProvider.GetRequiredService<ICustomerRepository>());
72      }
73
74      [Fact]
75      public void AddCommerX_GivesTheViewModelAndTheUseCaseTheSamePresenterInstanceWithinAScope()
76      {
77          using var provider = Build(ConfigurationWith(LocalDb));
78          using var scope = provider.CreateScope();
79
80          var outputPort = scope.ServiceProvider.GetRequiredService<ICreateCustomerOutputPort>();
81          var presenter = scope.ServiceProvider.GetRequiredService<CreateCustomerPresenter>();
82
83          Assert.Same(presenter, outputPort);
84      }
85
86      [Fact]
87      public void AddCommerX_ConfiguresTheDbContextWithSqlServerAndTheConfiguredConnectionString()
88      {
89          using var provider = Build(ConfigurationWith(LocalDb));
90          using var scope = provider.CreateScope();
91
92          var context = scope.ServiceProvider.GetRequiredService<CommerXDbContext>();
93
94          Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", context.Database.ProviderName);
95          Assert.Equal(LocalDb, context.Database.GetConnectionString());
96      }
```

```
97
98      [Fact]
99      public void TheShippedAppsettingsFile_IsValidAndHasTheConnectionString()
100     {
101         // appsettings.json se copia al directorio de salida junto con el host
102         var configuration = new ConfigurationBuilder()
103             .AddJsonFile(Path.Combine(AppContext.BaseDirectory, "appsettings.jso
    n"), optional: false)
104             .Build();
105
106         var services = new ServiceCollection();
107         services.AddCommerX(configuration);
108
109         Assert.False(string.IsNullOrWhiteSpace(configuration.GetConnectionString
    ("CommerX")));
110     }
111
112     [Fact]
113     public async Task DatabaseHealthCheck_WhenTheServerIsUnreachable_ReportsNoCon
    nectionInsteadOfThrowing()
114     {
115         // puerto 1 de la propia máquina: rechaza la conexión de inmediato
116         const string unreachable = "Server=127.0.0.1,1;Database=CommerX;User Id=
    x;Password=x;Encrypt=False;Connect Timeout=2;";
117         using var provider = Build(ConfigurationWith(unreachable));
118
119         var healthCheck = provider.GetRequiredService<DatabaseHealthCheck>();
120
121         Assert.False(await healthCheck.CanConnectAsync());
122     }
123 }
```

Qué verifica cada prueba:

WithoutTheConnectionString: si falta la configuración, el composition root falla al arrancar con un mensaje que nombra la cadena y el archivo.

BuildsAContainerWhereEveryRegistrationCanBeResolved: construye el contenedor con `ValidateOnBuild` y `ValidateScopes` (línea 36). La primera revisa que cada servicio registrado tenga todas sus dependencias; la segunda, que ningún Singleton consuma un Scoped. Es la prueba que te avisa si te olvidás de registrar algo.

ResolvesTheWholeChainOfACaseOfUse... y **GivesTheViewModelAndTheUseCaseTheSamePresenter...:** resuelven las cadenas reales dentro de un scope y confirman que presenter y puerto de salida son la misma instancia.

ConfiguresTheDbContextWithSqlServer...: el contexto usa SQL Server y la cadena configurada.
TheShippedAppsettingsFile...: el `appsettings.json` real se copia a la salida y es válido.
DatabaseHealthCheck_WhenTheServerIsUnreachable...: el chequeo informa "sin conexión" en lugar de lanzar una excepción.

```
dotnet test tests/CommerX.Desktop.Tests
```

Resultado esperado: las siete pruebas en verde. Si olvidaste el presenter de actualización (Módulo 8, Parte H), la prueba del contenedor falla con *"Unable to resolve service for type 'IUpdateCustomerOutputPort' while attempting to activate 'UpdateCustomerUseCase'"*: el mensaje te dice exactamente qué registro falta.

Checklist de verificación

Archivo / Tarea	Verificación
`CommerX.Desktop.csproj`	Proyecto WPF (`WinExe` , `net9.0-windows` , `UseWPF`) en la carpeta de solución `4.Infrastructure` ; referencias a InterfaceAdapter e Infrastructure; paquete `Microsoft.Extensions.Hosting` 9.0.20; `appsettings.json` con `CopyToOutputDirectory`
`appsettings.json`	Sección `ConnectionStrings` con la clave `CommerX` , barra invertida doble, apuntando a LocalDB
`CompositionRoot.cs`	`AddCommerX(IConfiguration)` falla con mensaje claro si falta la cadena; registra Application, InterfaceAdapter (los dos presenters y el ViewModel), Infrastructure, el chequeo y la ventana
`DatabaseHealthCheck.cs`	Recibe `IServiceScopeFactory` (no el DbContext), abre un scope por consulta y devuelve `CanConnectAsync()`
`MainWindow.xaml` y `.xaml.cs`	Recibe `DatabaseHealthCheck` por constructor; `InitializeComponent()` ; los dos manejadores delegan en `CheckAsync`

Archivo / Tarea	Verificación
`App.xaml` y `App.xaml.cs`	Sin `StartupUri`; hereda de `System.Windows.Application`; construye el host con `UseContentRoot`, resuelve la ventana del contenedor en `OnStartup` y detiene el host en `OnExit`
Ejecución	La ventana dice "Base de datos: conectada"; con una base inexistente dice "sin conexión" y no se cierra
`CommerX.Desktop.Tests`	Proyecto `net9.0-windows` que referencia al host; `dotnet test` con las siete pruebas en verde

Preguntas de reflexión

1. **¿Por qué el host es el único proyecto que referencia a todas las capas, y por qué ninguna otra capa lo referencia a él?**
   Pista: recordá la regla de dependencias. ¿Qué pasaría con el dominio si dependiera de la aplicación de escritorio?
2. **¿Qué pasaría si dejaras `StartupUri="MainWindow.xaml"` en `App.xaml`?**
   Pista: pensá quién crea la ventana en ese caso y qué recibe su constructor.
3. **¿Por qué `DatabaseHealthCheck` recibe `IServiceScopeFactory` y no el `CommerXDbContext`?**
   Pista: compará el tiempo de vida de un Singleton con el de un Scoped, y quién abre los scopes en una aplicación de escritorio.
4. **¿Por qué la cadena de conexión está en `appsettings.json` y no escrita en `CompositionRoot.cs`?**
   Pista: pensá en cuántos lugares habría que recompilar para cambiar de servidor, y en una contraseña dentro del código.
5. **¿Qué error habría detectado `ValidateOnBuild` si te hubieras olvidado de `AddUpdateCustomerPresenter()`, y cuándo lo habrías descubierto sin esa validación?**
   Pista: sin la validación, ¿cuándo se intenta construir el caso de uso de actualización?
6. **¿Para qué sirve `UseContentRoot(AppContext.BaseDirectory)`?**
   Pista: ¿dónde busca el host el archivo de configuración si no se lo decís, y desde dónde se puede lanzar un programa?

7. **Si mañana la aplicación pasara de SQL Server a MySQL, ¿qué líneas de este módulo tendrías que tocar?**
   Pista: buscá en el composition root la única línea que nombra a la infraestructura y en `appsettings.json` lo que depende del motor.